

xLLM的投机推理架构设计

演讲人：梁志伟

京东 / 算法工程师

AiCon
全球人工智能开发与应用大会

目录

01 投机推理加速原理

从自回归瓶颈到 draft / verify / rejection sampling, 解释为什么能加速且保持分布正确性。

02 投机推理技术演进

对比无模型、EAGLE / MTP / DFlash 等候选生成路径和系统代价。

03 xLLM投机推理架构设计

展开 runtime、worker-local loop、模型适配、KV cache 正确性与分布式执行。

04 投机推理未来展望

面向 Premium Latency 和 Mass Serving 的动态控制与成本权衡。

05 xLLM项目介绍

介绍 xLLM 系统架构、开源项目、GitHub 与用户群入口。

01 投机推理加速原理

原理与收益边界

自回归推理的核心矛盾

用户希望低延迟连续输出，但模型只能逐 token 串行生成

用户希望

服务体验像连续、稳定、实时的流。

○ **低首 token 延迟**
尽快看到第一个输出

○ **稳定 TPOT**
token 间隔稳定可预期

○ **连续流式输出 / 高并发**
多用户同时获得平滑体验

模型现实

自回归依赖决定：下一个 token 必须等待上一个 token 的采样结果。

Prefill 可并行

p1

p2

p3

p4

p5

p6

一次并行处理上下文

上下文作为整体进入 target model，计算密度相对更高。

Decode 必须串行

token_1

token_2

token_3

...

token_n

target
forward

target
forward

target
forward

target
forward

decode 阶段的系统代价

○ **Weight Read**
每步读取模型权重

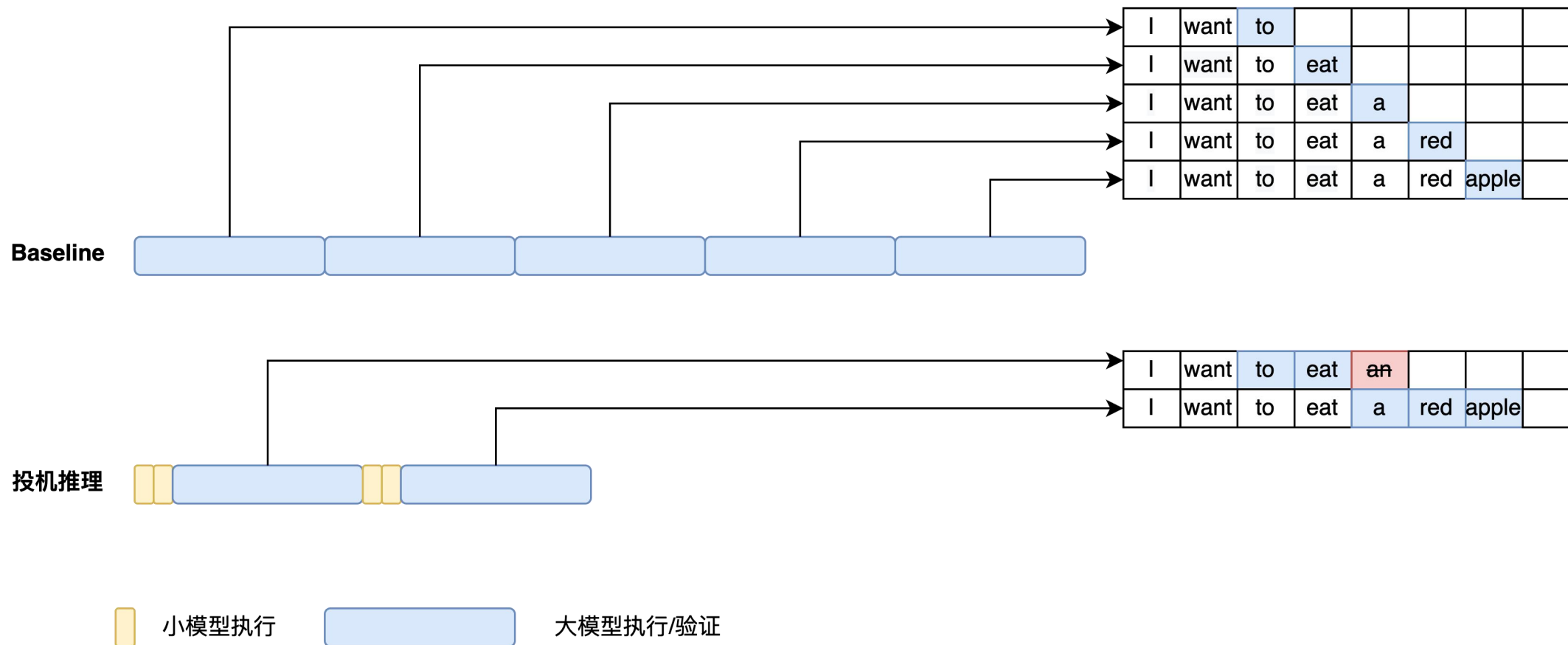
○ **KV Cache Read**
访问历史上下文状态

○ **Scheduler Wait**
并发排队与批处理等待

投机推理：一次验证多个 token

投机推理的加速原理

小模型快速起草候选token，大模型一次批量验证，最终输出仍由大模型保证



投机推理的过程

小模型先起草，再由大模型批量验证和纠错

- 1 小模型生成 draft tokens**
草稿模型基于当前上下文，快速连续生成多个候选 token。

当前上下文

want



Draft Model
低成本起草



draft tokens

to

eat

an

- 2 大模型验证 draft tokens**
主模型一次 forward 批量验证候选序列，连续匹配的 token 可直接接受。

context + draft tokens

want

to

eat

an



Target Model
一次批量验证



target outputs

to



eat



a



red



- 3 拒绝采样**
遇到首个不匹配 token 后，在拒绝点回退并重新采样。

连续匹配前缀

to

eat

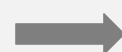


首个拒绝点

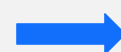
an

≠

a



Resample
回退后重采样



a

投机推理不是直接采用小模型输出，而是让小模型先起草，再由大模型验证和纠错。

如何保证模型精度

贪婪采样追求 token 完全一致；拒绝采样追求最终生成分布与 target model 对齐

A Greedy verification

确定性生成: temperature = 0

接受规则

draft x

==

target top-1



accept

● 保证目标

输出序列与 target greedy decode 一致。

● 工程含义

比较 target top-1 与 draft token。

token alignment

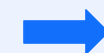
VS

B Rejection sampling

采样生成: temperature / top-p / top-k

接受概率

$a(x) = \min(1, p_{\text{target}}(x)/p_{\text{draft}}(x))$



$u \leq a(x)$

● 保证目标

最终分布与 target sampling 对齐。

● 拒绝之后

拒绝后按残差分布重采样

distribution alignment

拒绝后为什么不能直接从 p_{target} 再采样?

draft 分布已经贡献过一部分概率质量；拒绝点需从残差分布补齐 target 中尚未覆盖的部分。

target probability mass

draft contribution

residual fill

补齐
剩余概率



$p_{\text{residual}} \propto \max(0, p_{\text{target}} - p_{\text{draft}})$

采样模式下保证的是 distribution alignment, 不是逐 token 严格一致。

投机推理的收益边界

每轮多产出的有效 token，必须覆盖 draft 和系统额外开销

工程近似公式

$$\text{Speedup} \approx \frac{N_{\text{draft}} \times r_{\text{accept}}}{1 + N_{\text{draft}} \times c + \delta}$$

收益侧：平均接受长度
 $L_{\text{accept}} \approx N_{\text{draft}} \times r_{\text{accept}}$

成本侧：验证 + 起草 + 系统
 $1 + N_{\text{draft}} \times c + \delta$

一个直观例子

$N_{\text{draft}} = 4$
 $r_{\text{accept}} = 75\%$
 $c = 0.05, \delta = 0.10$
 $L_{\text{accept}} \approx 3$

Speedup

$\approx 2.31x$

$3 / (1 + 0.20 + 0.10)$

N_{draft}

draft 数
收益 ↑ / 成本 ↑

r_{accept}

接受率
决定 L_{accept}

c

草稿成本
过高会吃掉收益

δ

系统开销
通信 / 调度

加速成立条件

$\text{Speedup} > 1$



平均接受收益 > draft + 调度 + 通信 + KV 提交开销

低接受率

L_{accept} 上不去

重草稿模型

$N_{\text{draft}} \times c$ 太高

高系统开销

δ 压掉收益

投机式推理的目标不是“多猜”，而是“猜得划算”。

02 投机推理技术演进

候选token生成方式

关键不是“验证”本身，而是低成本、高接受率地产生候选 token

A 无模型 draft

候选Token来源

Context

History

Rules

代表方法

n-gram

prompt lookup

suffix decoding

● 优势

成本极低、接入简单、
无需模型管理。

● 局限

依赖重复模式，开放式
生成接受率不稳。

B 有模型 draft

候选来源

Draft Model

Feature Drafter

代表方法

SD

Medusa

EAGLE

MTP

DFlash

● 优势

质量更强、适用更广，可
训练提升接受率。

● 代价

额外计算、训练、通信与
调度复杂度。

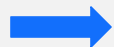
Target model verify

无论候选Token从哪里来，最终接受、拒绝、回退或重采样都由 target model 决定。

低成本猜测



高质量 drafter



动态选择最划算策略

成本、接受率、SLO 联合决策

候选生成的核心，是在成本与接受率之间选最划算策略。

无模型方式

利用上下文、历史请求、数据库中已出现的片段

1 Prompt / n-gram lookup

从 prompt 或已生成文本中寻找重复 token 片段。

repeated span -> continuation

代码补全 / 表格 / 模板化文本

2 Suffix / index lookup

用 suffix 匹配历史请求、上下文或轻量索引。

suffix match -> continuation

n-gram pool / token map / suffix automaton

3 RAG / 数据库

从检索 chunk、历史问答或业务模板抽取候选片段。

retrieved chunk -> candidate span

客服 / 知识问答 / 业务流程

来源抽取逻辑

Current signal

当前 n-gram /
suffix / query



Match source

上下文 / cache /
RAG



Candidate span

命中位置后的连续
token



Verify

交给
target

边界判断

● 优势

无额外模型权重、训练和 drafter KV。

● 风险

重复弱时，接受率下降。

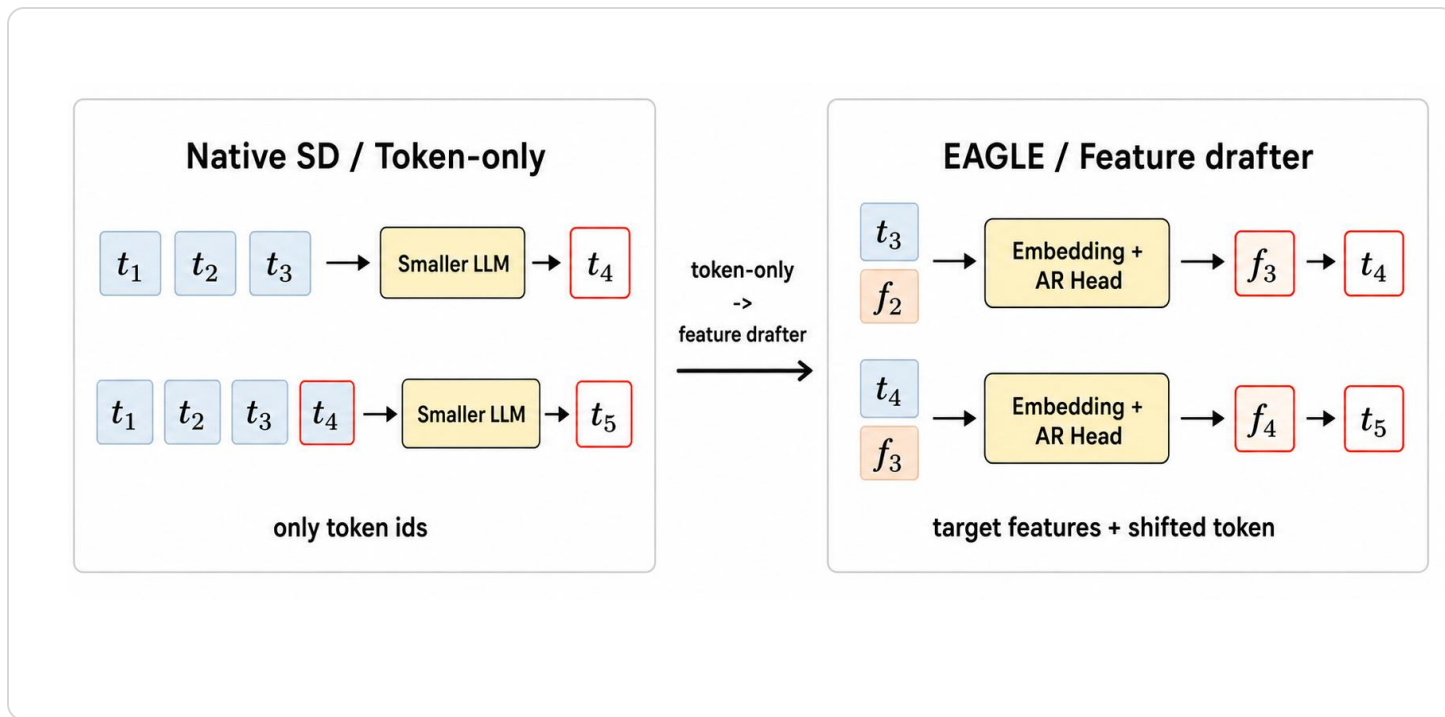
一句话结论

无模型 draft 不追求“猜得最准”，核心是让候选来源足够便宜。

没有额外模型和 drafter KV 开销；重复模式 / 检索命中明显时，可作为轻量优化或低成本 fallback。

EAGLE 系列

从复用 target feature 到模拟推理误差，核心是提升接受长度



EAGLE-3

让 drafter 更好适应推理误差传播。

- 1 Direct token target**
去掉间接的 feature 拟合约束。
- 2 Train/Test match**
训练时模拟回归误差，适应线上传播。
- 3 Multi-layer features**
融合多层 features，增强条件信息。

Native SD: 只看 token ids
接入简单，但看不到 target 内部状态

EAGLE: 复用 target feature
用 feature + shifted token 起草

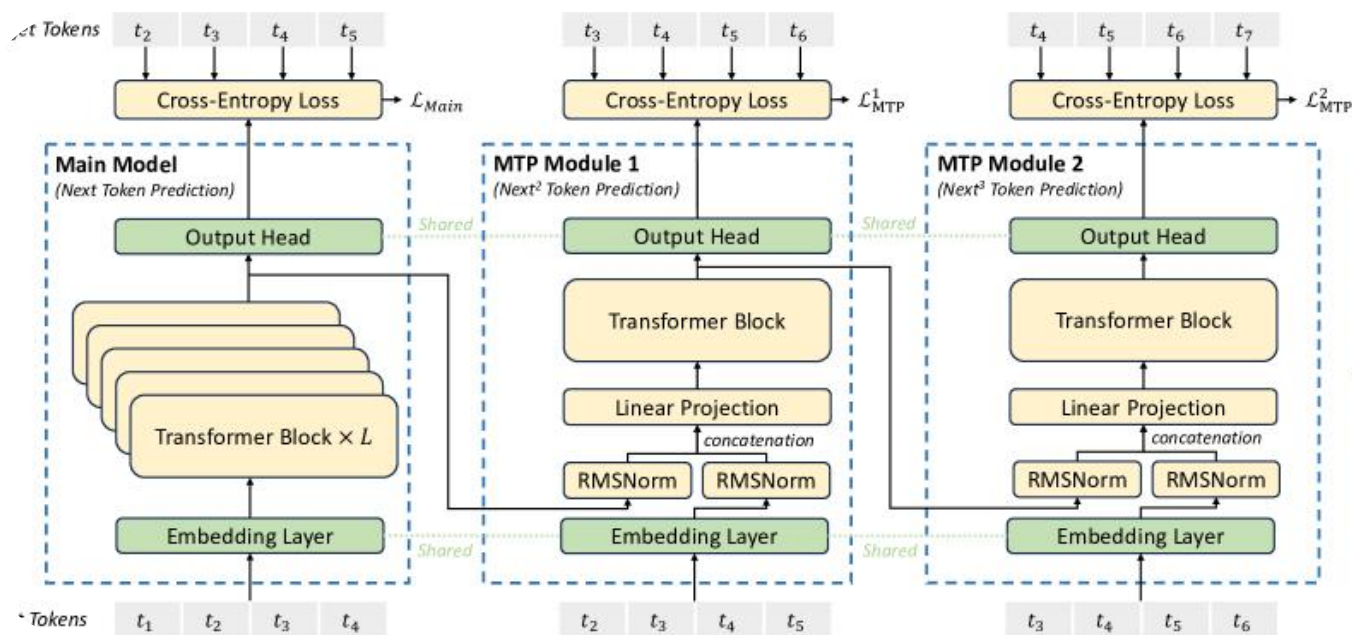
EAGLE-3: 贴近线上 verify
训练时模拟误差，稳定接受长度

EAGLE 系列让候选更接近 target；接受长度稳定后，可给更激进的 draft budget。

MTP: 模型自带的drafter

从“事后补一个 drafter”，转向“主模型训练时顺带学会多步预测”

DeepSeek-V3 MTP structure



机制变化

引入额外 MTP modules 预测更远未来 token。

- sequential module 结构保持因果链
- embedding / output head 与主模型共享
- MTP loss 作为训练期辅助目标
- 推理期作为 speculative decoding draft 来源

仍需 target verify

候选由 MTP 提出，正确性由 target 验证。

EAGLE: post-hoc drafter

target 训练后，围绕 feature 训练额外 drafter

MTP: native drafter

训练期内生多步预测，表示与输出更一致

Serving: verify + KV 提交

加载 modules，组织候选并提交 KV

DFlash: 并行生成

把多次自回归 draft 前向，变成 1 次并行生成

Draft-side latency model

AR drafter

context

t1

t2

t3

t4

自回归多次

DFlash

context

Mask tokens

block
diffusion

[t1, t2, t3, t4]

一次前向并行生成

系统要点

Dflash 的核心是压低 draft tokens 生成延迟。

- AR: draft 长度越长越慢
- DFlash: 一次前向生成 block
- target features 条件化 drafter
- KV Injection 增强条件强度

验证路径保持：接受最长有效前缀

Target feature

提供未来上下文，不只依赖 token ids

KV Injection

把 target feature 注入每层 KV

Block diffusion

并行预测 block，减少串行等待

03 xLLM投机推理架构设计

运行时架构设计

xLLM 团队：面向国产芯片的 LLM 推理引擎

国产芯片、多模型形态与企业级部署场景，是投机推理落地的系统基座

01 国产芯片支持

昇腾对比 vllm-ascend / MindIE：1.7X-2.2X

已支持昇腾、寒武纪、摩尔线程、天数智芯等国产芯片

02 国产模型与多模态支持

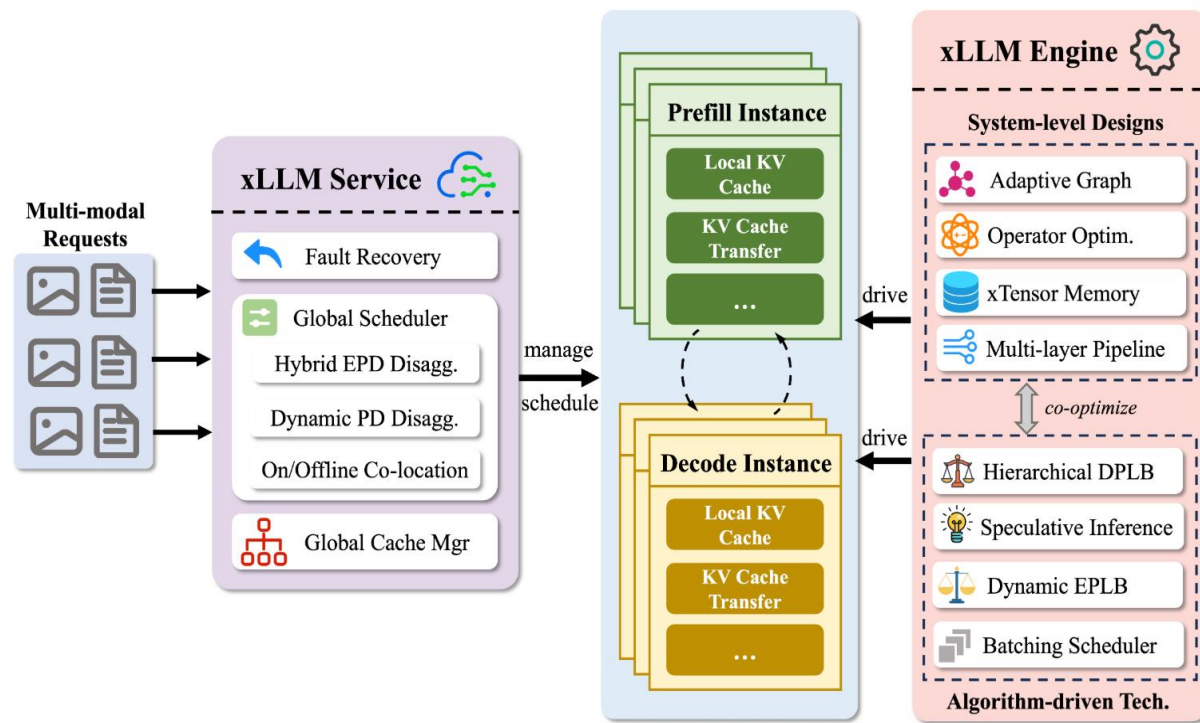
智谱 GLM 默认使用 xLLM 作为国产化芯片推理引擎

支持 DeepSeek、Qwen、GLM 及多模态 / 生成式推荐

03 开源推广

开源 10 个月，GitHub star 1.3k

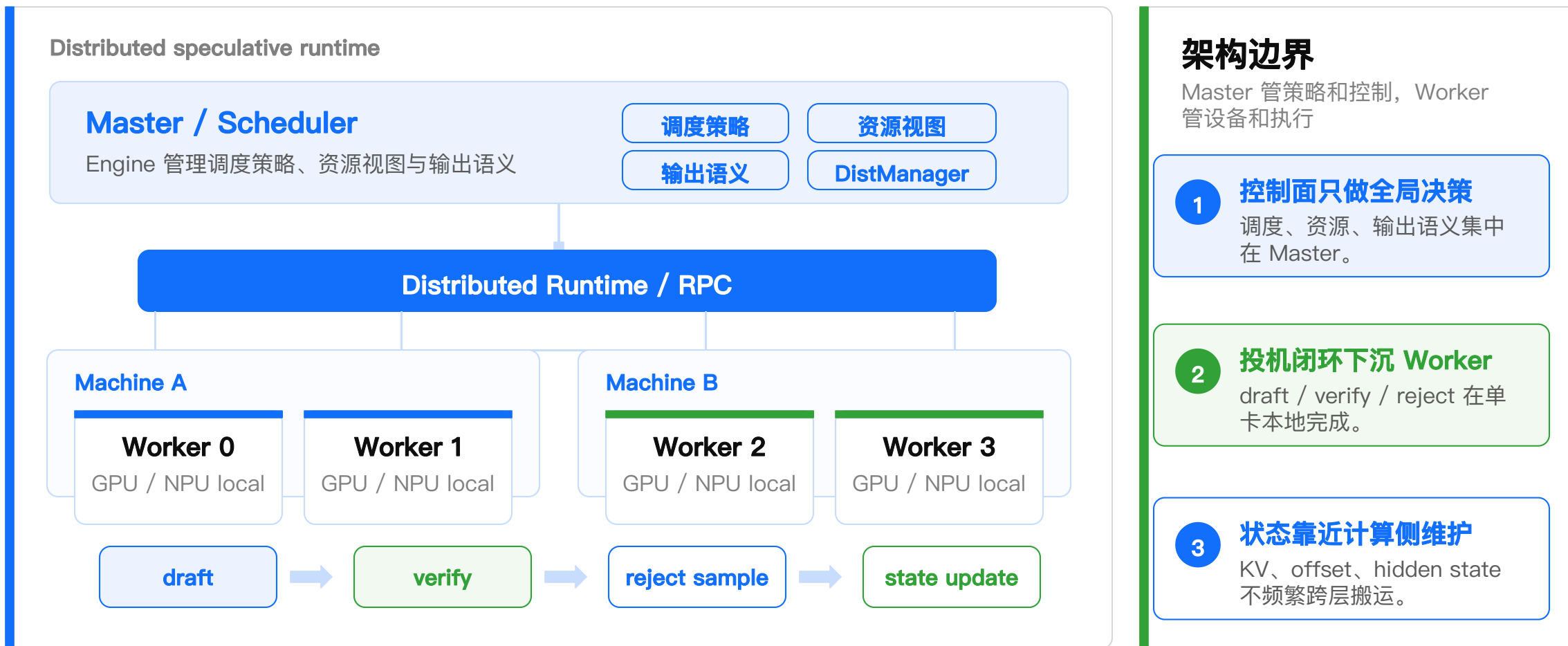
覆盖国企、运营商、智算中心、医院、高校、政府等



GitHub 全面开源：<https://github.com/jd-opensource/xllm>

xLLM 投机推理运行时架构

控制面负责全局调度，Worker-local runtime 承接高频投机闭环



xLLM 的核心架构选择：Master 管策略，Worker 管高频投机执行，把跨层状态搬运压到最低。

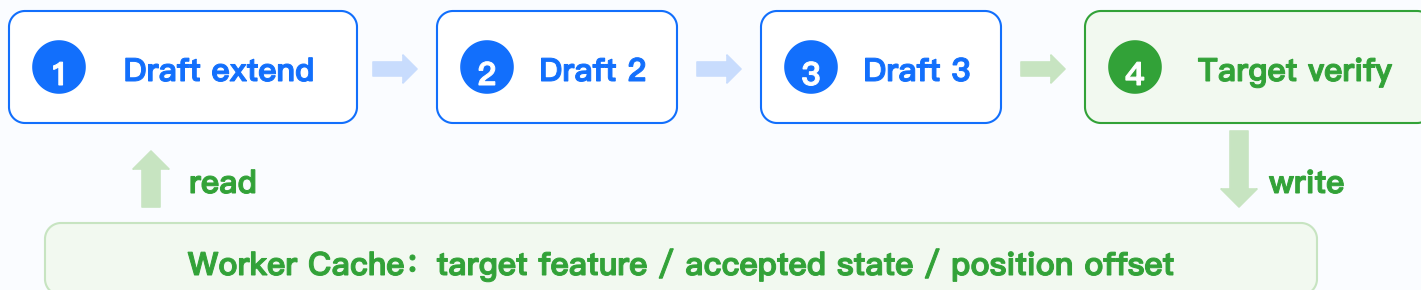
xLLM Worker-local Decode Step

单 step 内完成投机推理，为异步 overlap 提供稳定边界

Simplified reference flows



xLLM 单 step: 投机推理执行顺序 + cache 更新逻辑



单 step 闭环收益

把细粒度执行封装成稳定的 step 结果

1 **执行顺序本地可控**
draft 多步与 target verify 在 Worker 内串联。

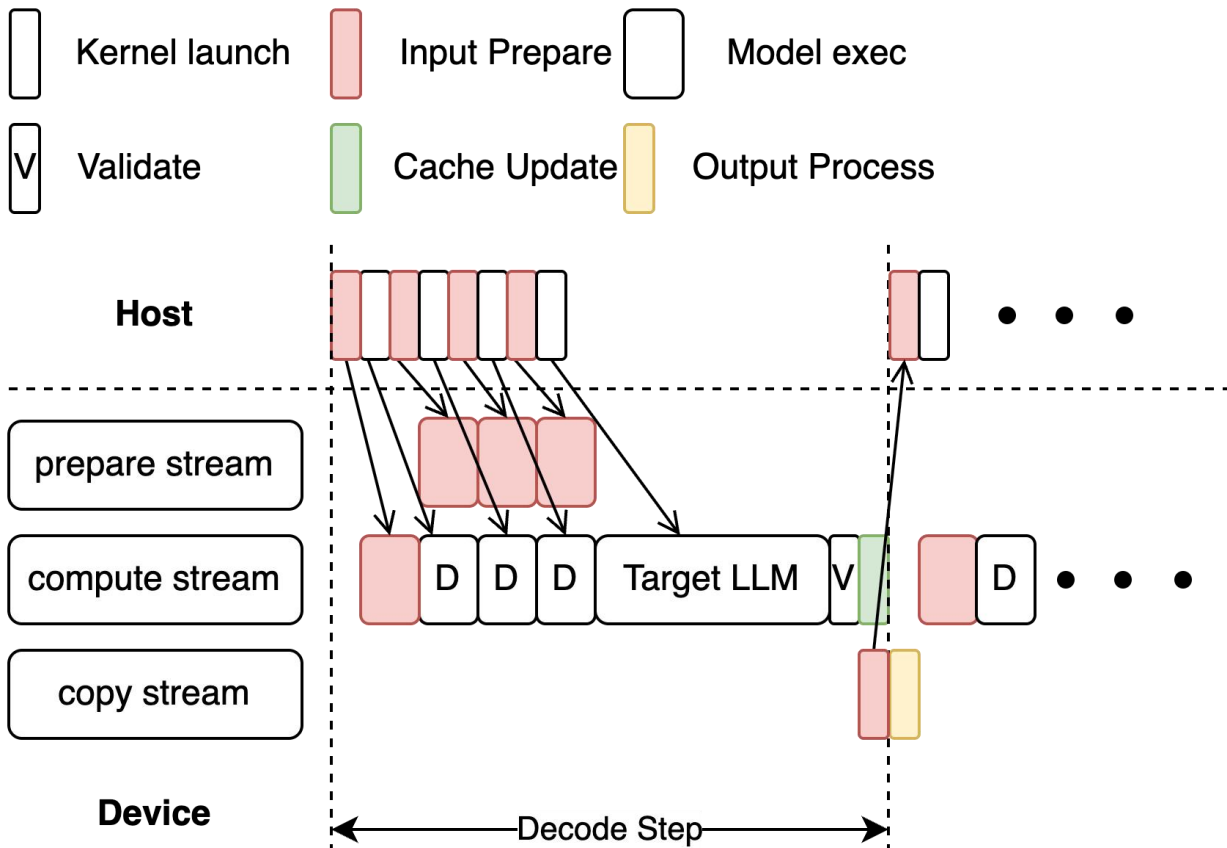
2 **Cache 更新边界清晰**
accepted length / position / KV 在本地提交。

3 **为异步 overlap 铺路**
上层只感知 step 结果，不阻塞内部推进。

异步流水线的设计

用模型计算掩盖投机推理带来的 host 开销

原始异步流水线图



读图要点

Stream 职责

1

```
prepare stream 准备 metadata / H2D;  
compute stream 执行 draft / verify。
```

Event 边界

2

metadata_ready_event 连接 prepare / compute, ready_event 延迟回收。

依赖不并行

3

draft extend -> draft 2 -> draft 3 -> verify 仍按 token 串行。

收益来源

4

上一轮 D2H 与下一段输入准备，被本轮模型计算掩盖。

大规模分布式部署适配

PD 交接状态, Decode 侧按 Attention / MOE 分层处理

Distributed speculative adaptation

Prefill Node

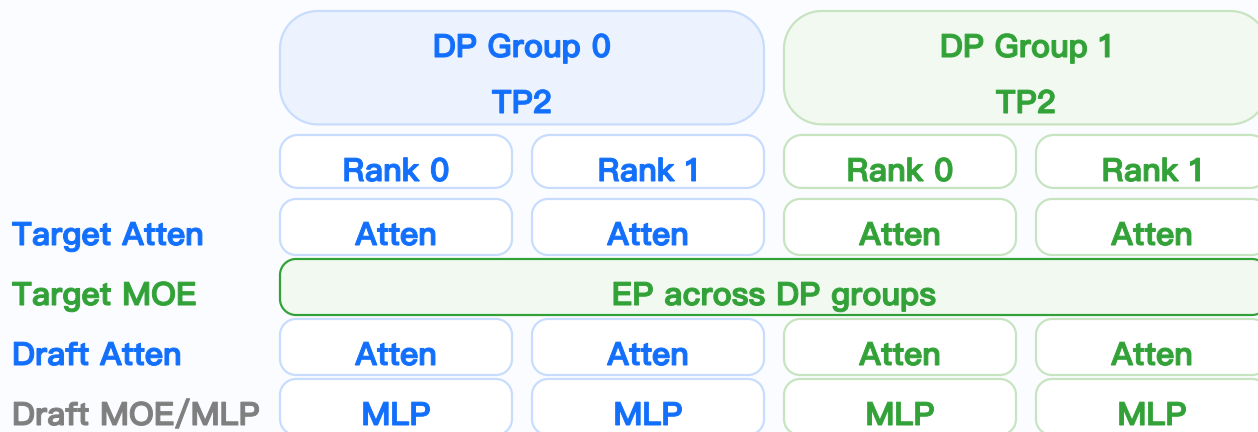
先 target 后 draft, 只准备可接续状态

Target prefill

Draft prefill

PD Handoff / KV Transfer: target KV + draft KV + metadata

Decode Node



适配边界

Decode 按组件承接并行

1 Prefill 适配

target → draft, 只准备状态。

2 Attention

target / draft 均按 DP rank 本地处理。

3 MOE / MLP

target MOE 用 EP; draft MLP 本地。

核心结论

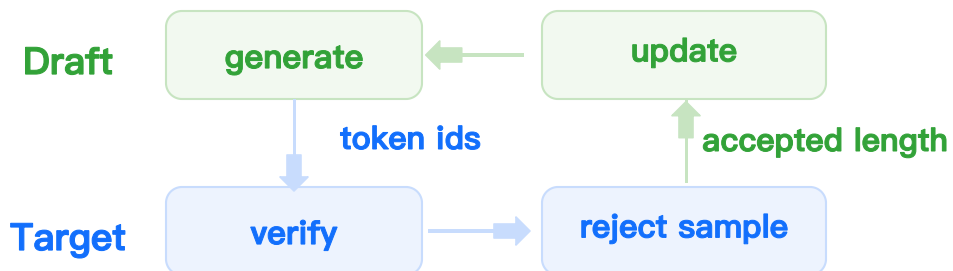
按组件适配, 并保证 target / draft cache 正确性。

模型适配

标准 KV Cache 类型的正确性适配

两套 KV Cache 域分离

draft cache 负责候选生成，target cache 负责验证后的真实状态提交；两者不共享 KV block，只通过 token ids 和 accepted length 对齐。



正确性目标

两套 cache 都必须回到同一个真实状态：target 保证语义正确，draft 保证下一轮候选起点正确。

01 Full / PagedAttention

完整历史 KV 可见，PagedAttention 主要改变分页与复用方式。

block table

slot mapping

kv_seq_lens

- target: 拒绝位覆盖，accepted length提交
- draft: 处理方式与target相同

02 Sliding Window Attention

保证double buffer的窗口起点、逻辑可见范围和写指针一致。

rolling KV

window start

write pointer

- target: verify 后推进窗口，reject 后恢复边界
- draft: 与target相同

标准 KV 类型的适配重点：两套 cache 域分离，但都必须按 accepted length 对齐到真实前缀。

模型适配

复杂 State / 压缩 KV 类型的正确性适配

01 Linear Attention

recurrent state 替代完整历史 KV



状态

Linear states

边界

linear_state_id

动作

select state

target / draft state 分离：暂存验证过程中每个token对应的cache，最后按 accepted length 选择。

02 CSA / DSA

local KV + compressed KV + sparse metadata



状态

c4 / c128 state

边界

compress size

动作

commit rows

验证过程产生的token cache先缓存，拒绝采样后，根据accepted length 更新 compressed state。

03 DFlash

特殊 draft 结构，不是 attention 类型



状态

draft state

边界

slot mapping

动作

update cache

block cache 依赖 target 输出的特征。拒绝采样后，需根据接受长度使用 target feature重建draft kv cache。

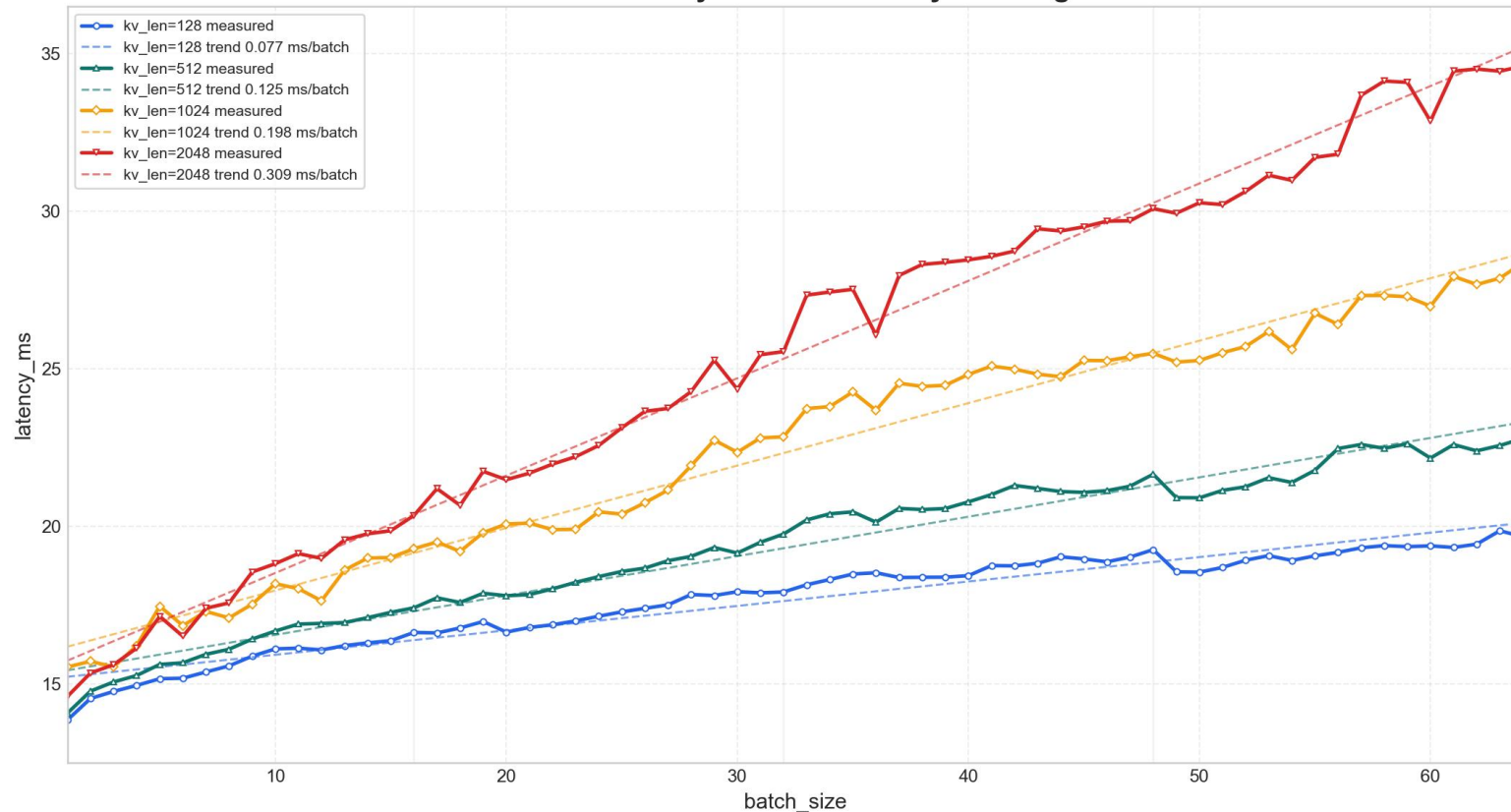
复杂模型适配的重点：把不同的Attention 类型纳入统一 cache 正确性边界。

高并发下投机推理为什么会劣化

batch 越大, baseline 固定开销越容易被摊薄, 投机边际收益越小

Decode latency trend under batching

Decode Latency vs Batch Size by KV Length



并发下的收益变化

收益边界会随 batch 区间重写

1

低并发 / 小 batch

baseline 每 token 付固定开销; 投机一次 verify 覆盖多 token, 更容易摊薄成本。

2

中等 batch

看省掉的固定开销, 能否覆盖 draft / verify / 调度 / KV 额外开销。

3

高并发 / 大 batch

continuous batching 已摊薄固定开销; 投机额外计算开始挤占正常 decode。

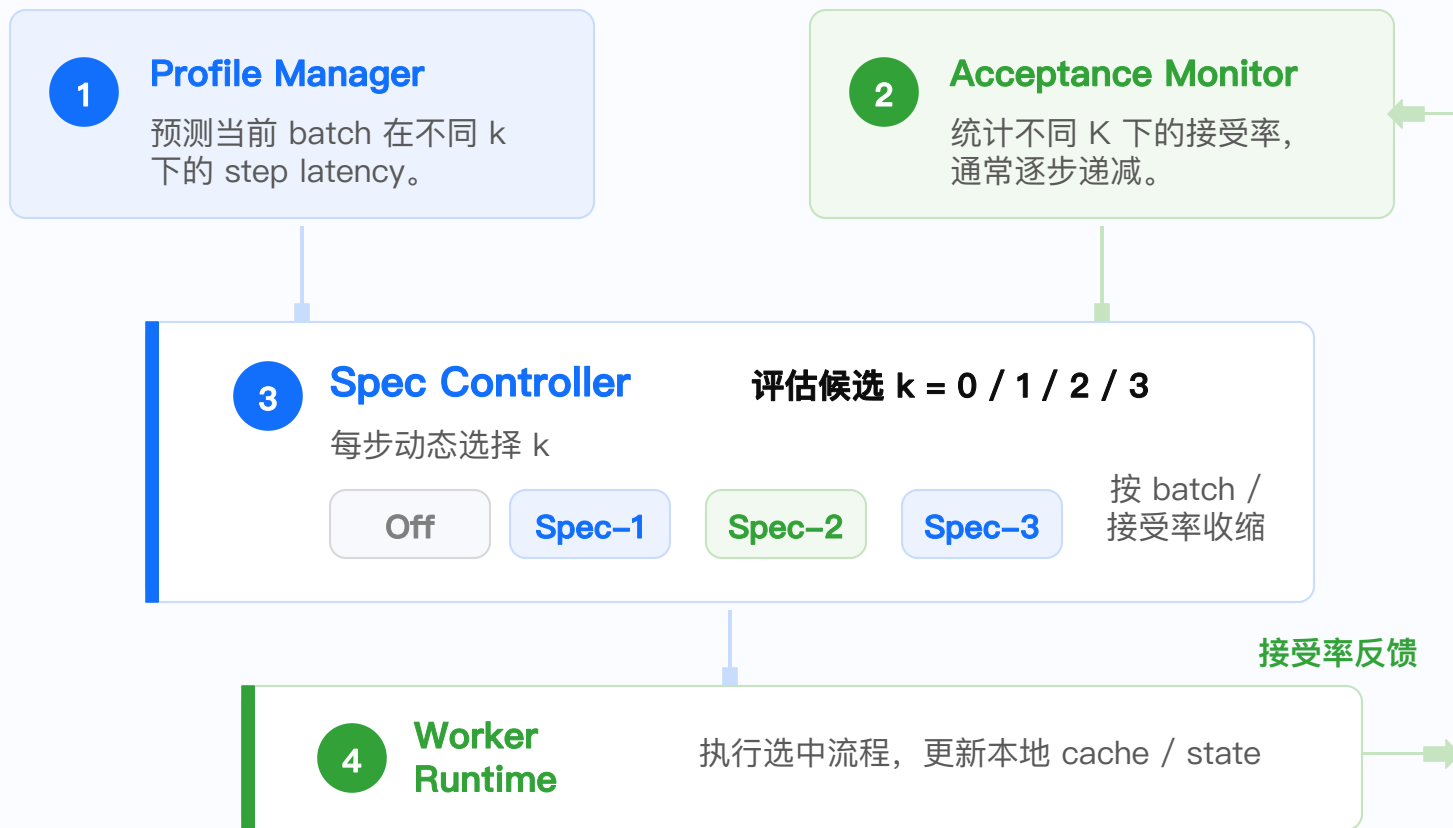
临界点判断

固定开销节省接近投机推理的额外开销时, 加速比接近 1; 并发继续增大后容易转为吞吐劣化。

动态投机推理控制

每个 decode step 动态选择是否投机，以及投机几步

xLLM 动态控制闭环



控制目标：刚好值得猜

batch 越大，k 越短；减少临时 KV 与验证回滚开销

小 batch

Spec-3



固定开销占比高，长 draft 更容易摊薄。

中 batch

Spec-2 / 1



收益接近临界点，按预测结果动态缩短。

大 batch

Off / 1



减少被拒绝 token，保护正常 decode 吞吐。

04 投机推理未来展望

策略与未来展望

投机推理的两个发展方向

一条用更多预算压低 TPOT，一条用动态投机预算提升集群效率

01 极致低延迟

TPOT / 端到端时延优先

面向高价值交互，用更多预算压低 TPOT 和可感知等待。

目标 降低 TPOT、端到端 latency 和用户可感知等待时间。

场景 高价值交互、Agent、Coding、语音对话、实时问答。

策略 更强 drafter、更激进 overlap、更少同步和更短执行路径。

取舍 可接受更高单位 token 成本，但需要按用户等级 / 业务 SLO 定价。

关注指标

TPOT

E2E

P90

02

MASS 平台效率 动态投机预算 / 单位成本优化

面向默认平台服务，动态分配预算并守住 SLO。

目标 提高 GPU/NPU 利用率，降低单位 token 成本，守住 TPOT / TTFT / 吞吐 SLO。

场景 默认平台服务、混合负载、多租户平台、成本敏感业务。

策略 低负载填充空闲算力；高负载收缩 draft length、降低 verify budget 或关闭投机。

取舍 不追求单请求极致加速，优先保证 Goodput、SLO 达标率和资源利用率。

关注指标

Goodput

SLO

Cost

共同底座

Runtime / Scheduler

统一收益判断与资源放置

输入信号

负载

接受率

KV 水位

SLO



策略动作

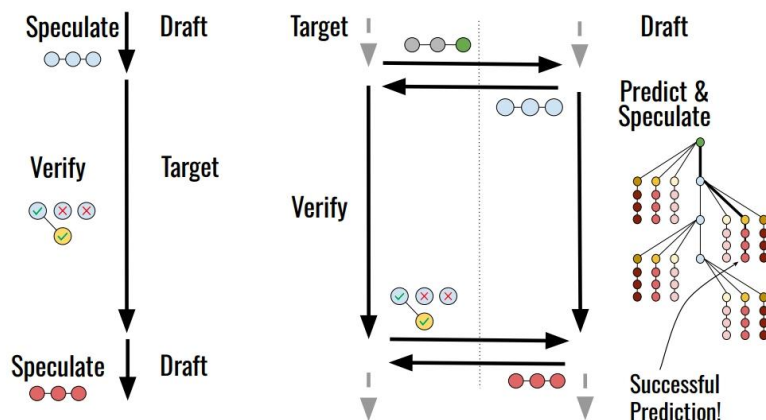
选择 drafter / draft length
verify budget / fallback

极致低延迟：前移下一轮准备

在 target verification 期间准备下一轮 draft，继续压缩等待链路

Speculative Decoding / Saguaro

普通 SD 等验证结果确定后再开启下一轮；SSD 把下一轮准备前移到 verify 期间。



Predict ahead

Hide overhead

极致低延迟

命中预测集合时，下一轮 draft 立即接上。

快速通道的三类动作

重点不是最低单位成本，而是用更多并行、预测和调度复杂度换低 TPOT。

SSD

提前准备下一轮

target verify 执行时，drafter 预测接受路径并准备下一轮 speculation。

D-Flash

缩短 draft 阶段

用 diffusion / block generation 生成 draft block，减少自回归串行时间。

PEARL

减少互等

用 pre-verify / post-verify 让 draft 与 verify 局部重叠。

结论

更适合高价值请求的极致低延迟通道；默认平台服务高负载下仍需动态收缩或降级。

Fast path only

MASS 平台效率：动态投机预算

按负载、请求、token 与 SLO 动态分配投机预算

01 Goodput / 负载闭环

SmartSpec / TurboSpec · Nightjar

- 用有效 token 吞吐判断投机收益，不只看单请求 latency。
- 按 batch、接受率和排队压力调 draft length；高负载关闭。

02 Step 内动态调度

FASER

- continuous batch 内动态选择 speculation length。
- verify 拆成 chunks，与后续 draft overlap，减少等待。

03 Verification Budget 分配

ECHO · TETRIS

- 高并发下 verification compute 稀缺，不能平均分给候选。
- 按置信度、接受概率和 depth / width 收益选择路径。

04 SLO-aware / 在线策略选择

AdaSpec · TapOut · BanditSpec

- 先保障紧急请求 TPOT / SLO，再用剩余预算提升 Goodput。
- 按 prompt、任务分布、drafter 表现和反馈选择停止策略。

共同趋势

从固定开关 / draft length，走向按负载、请求和 token 动态分配预算。

xLLM 落点

统一成本模型：Goodput / SLO / batch / 接受率 / draft cost / KV / verify budget。

Goodput

SLO

KV 水位

Budget

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO

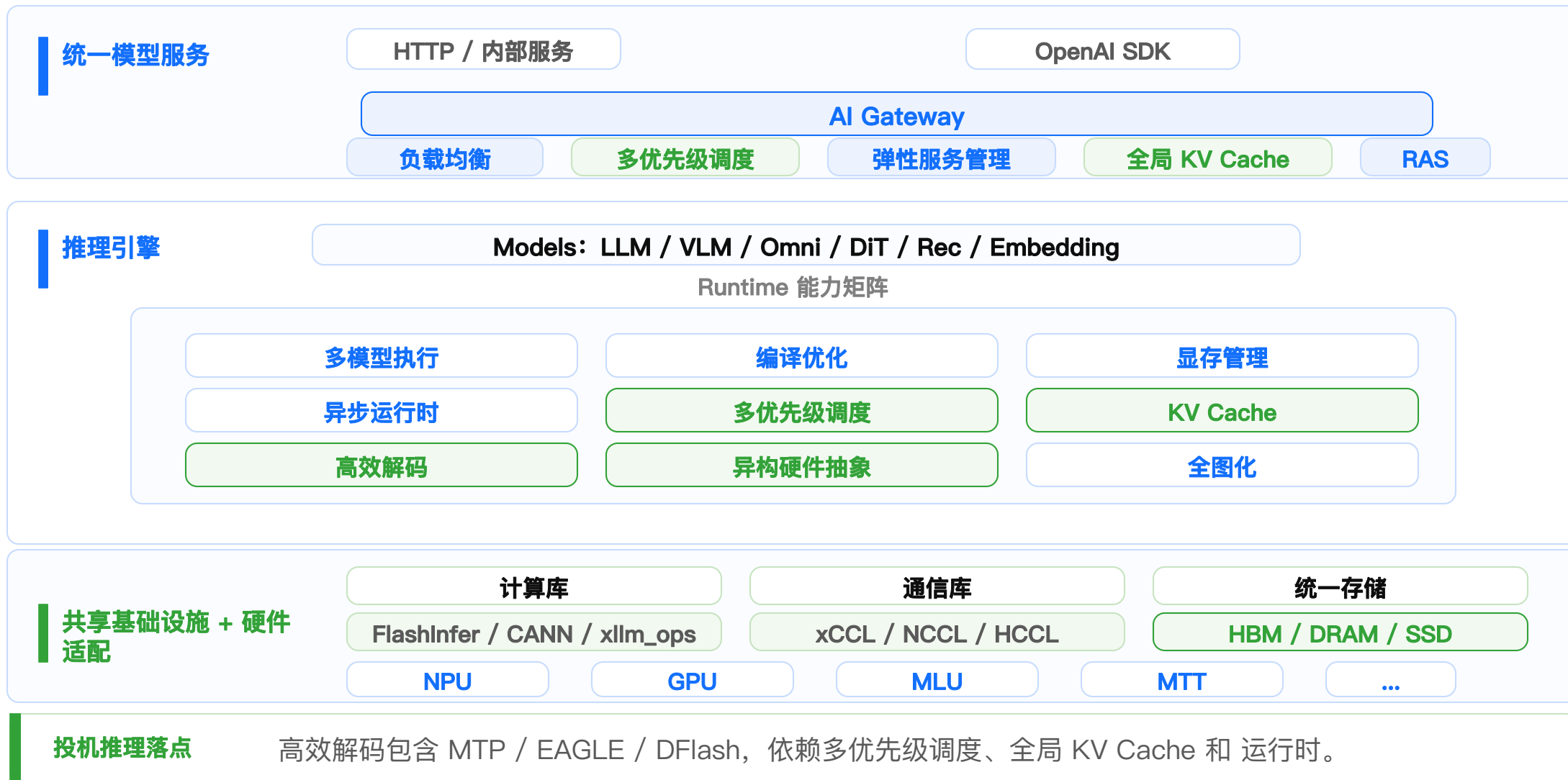


05 xLLM项目介绍

架构与开源生态

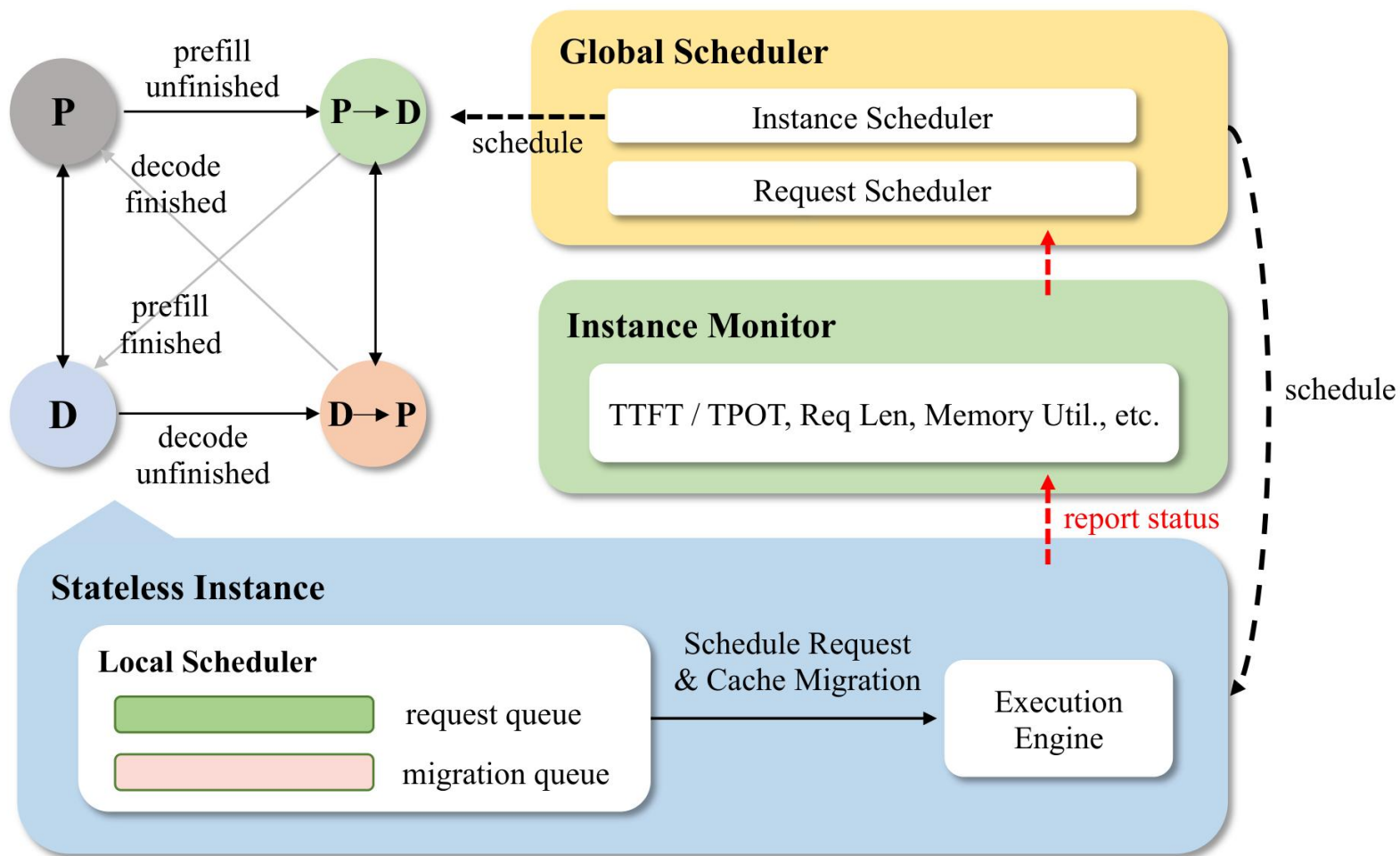
xLLM 架构总览

从服务入口、统一模型服务到推理引擎、共享基础设施和异构硬件适配的系统栈



Dynamic PD: SLO-aware 资源切换

静态 PD 难以适应负载波动; Dynamic PD 按实时 SLO 弹性切换实例角色



调度闭环看什么

把 P/D 作为请求级属性, 由 SLO 与资源指标驱动角色切换。

1 实时压力

采集队列、请求长度、显存和时延。

2 切角色

TTFT 高: D→P; TPOT 或 running tokens 高: P→D。

3 选实例

Prefill 选最短队列; Decode 优先复用实例。

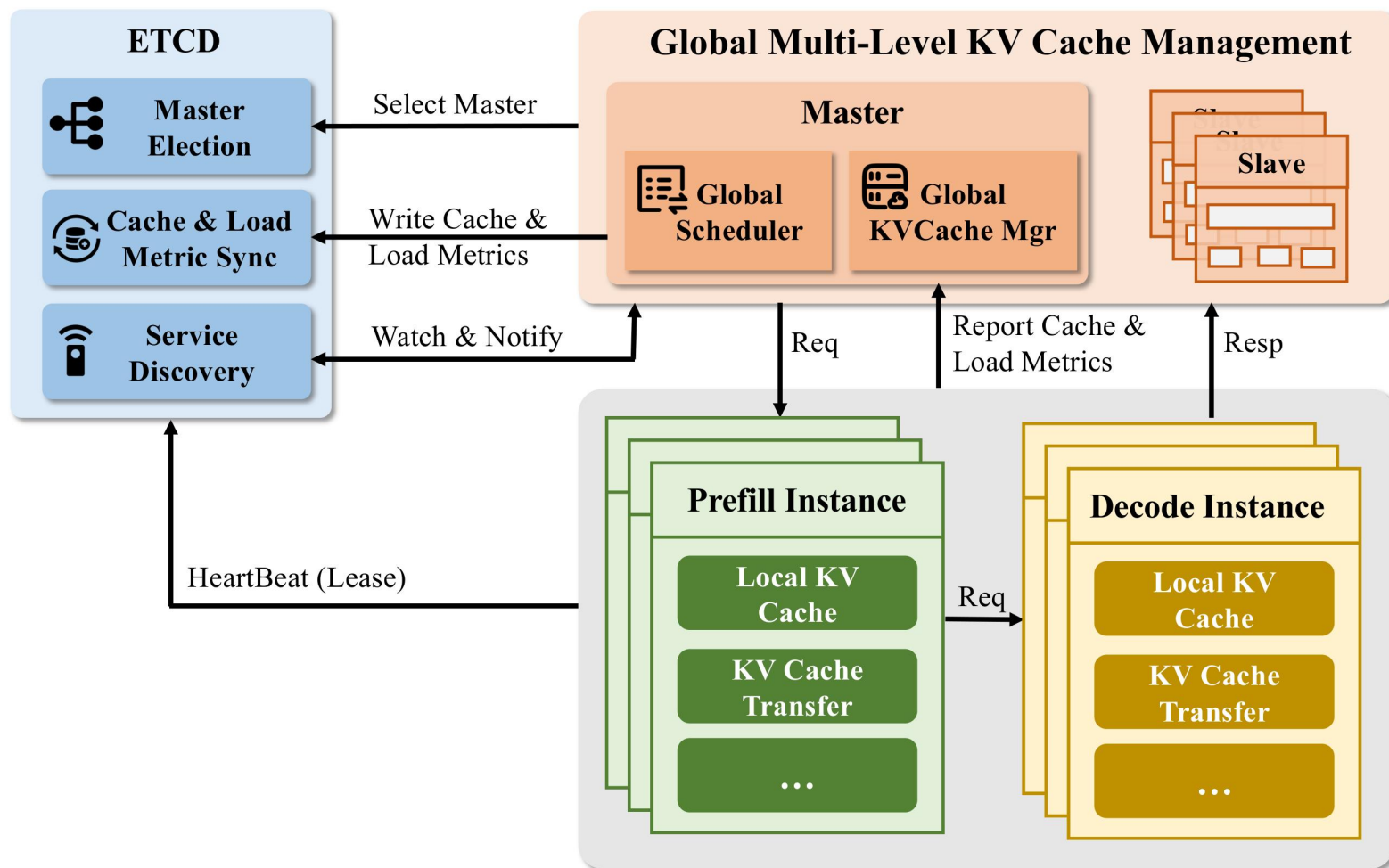
Elastic Pools



静态比例 → SLO-aware

全局多级 KV Cache 管理

长上下文和 PD 分离让 KV Cache 变成跨实例、跨层级、跨网络的全局状态管理



KV 调度看什么

把 KV 位置、迁移成本、实例负载和 SLO 纳入同一决策。

1 为什么要全局 KV

长上下文反复读历史 KV，显存/带宽成为瓶颈。

2 全局层怎么管

ETCD 选主和发现；Master 维护元数据并统一调度。

3 数据面怎么跑

迁移引擎搬迁；本地缓存做冷热分层。

Routing Signals

Prefix

Load

Cost

命中率 / 迁移成本 / SLO

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会